

CSCI 650 - Algorithms and Computability

Assignment 4

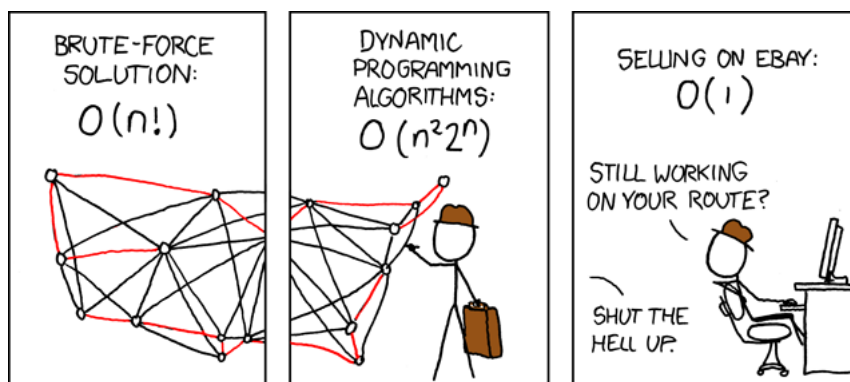
Solutions to the written questions on this assignment should be submitted via PDF to Canvas before **October 13th at 11:59 pm**. Implementations for question 1 should be submitted to INGINious by the same deadline. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions **independently**. Feel free to communicate via [Discord](#) and to post questions on the appropriate forum in Canvas. Do not post solutions.

Have fun!

Problems

1. In this problem we will develop and analyze two heuristic approaches to solving the [Traveling Salesman problem](#), one using nearest neighbors and the other using simulated annealing.



<https://xkcd.com/399/>

- (a) (5 pts) Write a function `totalDistance(D, path)` which returns the total distance a traveling salesman must travel given a sequence of cities. `path` will include each city exactly once, note that the salesman must end at the start city. Submit your code on INGINious and include a code snippet of this function in your PDF submission.
- (b) (10 pts) Write a function `nearestNeighbor(D, start)` which returns an order in which to visit cities with associated distance matrix `D` starting at `start` and following the nearest

neighbor heuristic. Submit your code on INGIInious and include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function?

- (c) (5 pts) Use the `tspTests.py` script provided on Canvas to generate histograms depicting the distribution of ratios (observed distance)/(minimum observed distance) under different circumstances. Describe what you observe.
- (d) (5 pts) Write three functions to generate candidate solutions.
- `swapPairs(path, numSwaps, adj)` swaps `numSwaps` random pairs of elements in `path`. If `adj` is `True`, only swap adjacent pairs of elements in the current path.
 - `permuteRange(path)` randomly permutes a slice of `path` between two randomly selected indices.
 - `reverseRange(path)` reverses a slice of `path` between two randomly selected indices.

Include code snippets of these functions in your PDF submission.

- (e) (10 pts) Write a function `runSimulatedAnnealing(D, genCandidate, maxIterations, T, alpha)` which finds a path for a traveling salesman using simulated annealing given a distance matrix `D`, a function to generate candidate solutions, a maximum number of iterations to perform, an initial temperature, and a multiplicative update factor for temperature. The initial path should be a random permutation of the cities. Include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function with respect to n (the number of cities) and i (the number of iterations)?
- (f) (5 pts) Use the `tspTests.py` script provided on Canvas to generate plots showing how the total distance of the current solution changes over time. Discuss the results and briefly describe a process for tuning the hyperparameters of this approach.

2. Let us return to the busy beaver numbers.

- (a) (5 pts) Describe why the busy beaver numbers are incomputable intuitively. What are the practical consequences of this with respect to determining $BB(n)$?
- (b) (10 pts) Show that the busy beaver numbers are incomputable via reduction from a problem discussed in class. In particular, write pseudocode (using Python syntax) showing that a known undecidable problem can be solved assuming the existence of a function $BB(n)$ which is guaranteed to halt.